

Université Hassan II de
Casablanca
Faculté des Sciences & Techniques
Mohammedia

Licence Informatique, Réseaux & Multimédia

Collecte de Données Multimodales

Réalité Image

Guide Complet des Mécanismes d'Acquisition des Images

Réalisé par :
Prof. Abdellah ADIB

May 1, 2026

Contents

1	Introduction	2
1.1	Contexte et Objectifs	2
1.2	La Realite Image	2
1.3	Vue d'Ensemble des Mecanismes	2
2	Capture d'Ecran	4
2.1	Principe Fondamental	4
2.1.1	Architecture de la Capture	4
2.1.2	Methodes de Capture	4
2.2	Capture avec Pillow (PIL)	4
2.2.1	Capture Regionale	5
2.3	Capture Haute Performance avec MSS	6
2.3.1	Capture en Boucle (Monitoring)	7
2.4	Capture avec PyAutoGUI	8
2.5	Bonnes Pratiques	8
3	Camera et Webcam	9
3.1	Principe Fondamental	9
3.1.1	Architecture de Capture Video	9
3.2	Capture avec OpenCV	9
3.2.1	Capture Simple d'une Frame	9
3.2.2	Capture Video en Direct	10
3.2.3	Enregistrement Video	11
3.3	Configuration Avancee	13
3.3.1	Parametres de la Camera	13
3.4	Detection de Visages en Temps Reel	13
3.5	Bonnes Pratiques	14
4	Scanner	16
4.1	Principe Fondamental	16
4.1.1	Types de Scanners	16
4.2	Protocoles de Communication	16
4.2.1	TWAIN (Windows/macOS)	16
4.2.2	SANE (Linux)	17
4.3	Scan Multi-Pages (ADF)	18
4.4	Post-Traitement des Scans	20
4.4.1	Redressement (Deskew)	20
4.4.2	Suppression du Bruit	21
4.5	Bonnes Pratiques	22
5	Telechargement d'Images	23
5.1	Principe Fondamental	23

5.2	Telechargement Simple avec Requests	23
5.2.1	Telechargement Multiple	24
5.3	Telechargement Asynchrone	25
5.4	APIs d'Images	26
5.4.1	Unsplash API	26
5.4.2	Pexels API	27
5.5	Scraping d'Images	28
5.6	Bonnes Pratiques	29
6	Generation d'Images par IA	30
6.1	Principe Fondamental	30
6.1.1	Modeles Principaux	30
6.2	Stable Diffusion (Local)	30
6.2.1	Parametres Avances	31
6.3	DALL-E 3 via OpenAI API	32
6.4	Image-to-Image	33
6.5	Inpainting	34
6.6	Bonnes Pratiques	35
7	Extraction de Frames Video	36
7.1	Principe Fondamental	36
7.2	Extraction avec OpenCV	36
7.2.1	Extraction Selective	37
7.2.2	Extraction par Intervalle Temporel	38
7.3	Detection de Changements de Scene	39
7.4	Creation de Mosaïque	41
7.5	Bonnes Pratiques	42
8	Conclusion	43
8.1	Tableau Comparatif des Mecanismes	43
8.2	Bonnes Pratiques Generales	43
8.3	Arbre de Decision	44
9	Bibliographie et Ressources	45
9.1	Ouvrages de Reference	45
9.2	Documentation Officielle	45
9.3	Tutoriels Recommandes	45
9.4	Jeux de Donnees pour Pratiquer	46
9.5	Aspects Legaux et Ethiques	46

Chapitre 1 Introduction

1.1 Contexte et Objectifs

La collecte de données constitue la première étape fondamentale de tout projet d'intelligence artificielle et d'apprentissage automatique. Sans données de qualité, même les algorithmes les plus sophistiqués ne peuvent produire de résultats pertinents. Ce rapport se concentre sur la **Realite Image** de la collecte de données, c'est-à-dire l'ensemble des mécanismes permettant d'acquérir des images exploitables par un système informatique.

1.2 La Realite Image

Le concept de **Realite Image** désigne tous les procédés techniques permettant de capturer, générer ou obtenir des images sous forme numérique. Contrairement à la simple visualisation d'images, la *Realite Image* concerne l'acquisition systématique et programmable d'images dans un format exploitable (PNG, JPEG, TIFF, BMP, WebP, etc.).

Les 6 mécanismes principaux de la Realite Image sont :

1. **Capture d'Écran** : Capture programmatique de l'affichage graphique
2. **Camera / Webcam** : Acquisition en temps réel depuis un périphérique vidéo
3. **Scanner** : Numérisation de documents et photographies physiques
4. **Telechargement** : Obtention via HTTP/HTTPS ou APIs
5. **Generation IA** : Création synthétique avec Stable Diffusion, DALL-E, Midjourney
6. **Extraction Video** : Extraction de frames depuis fichiers vidéo

1.3 Vue d'Ensemble des Mécanismes

Chaque mécanisme répond à des besoins spécifiques :

- **Capture d'écran** : Automatisation, tests, monitoring, surveillance système
- **Camera/Webcam** : Vision par ordinateur, reconnaissance faciale, sécurité
- **Scanner** : Archivage, OCR, préservation de documents historiques
- **Telechargement** : Datasets ML, scraping visuel, agrégation de contenu
- **Generation IA** : Création de contenu, data augmentation, prototypage visuel
- **Extraction video** : Analyse temporelle, détection d'événements, timelapses

Tous ces mécanismes convergent vers un objectif commun : obtenir des données visuelles sous forme de tableaux de pixels (arrays numpy) ou fichiers images standardisés, indépendamment du mécanisme utilisé.

Conseil

Choisissez le mecanisme en fonction de la **source** de l'image :

- Source = Ecran ? → Capture d'écran
- Source = Monde reel ? → Camera/Scanner
- Source = Web ? → Telechargement
- Source = Imagination ? → Generation IA
- Source = Video ? → Extraction de frames

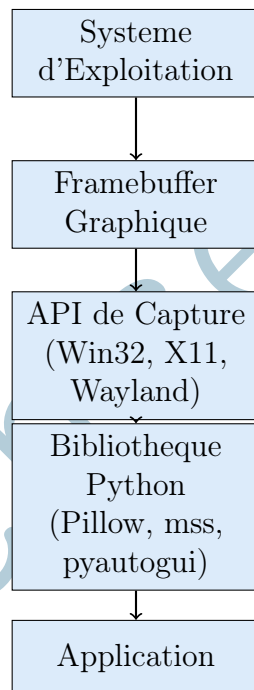
Licence IRM

Chapitre 2 Capture d'Ecran

2.1 Principe Fondamental

La capture d'écran consiste à capturer programmatiquement le contenu affiché sur un écran. Contrairement à une simple pression sur la touche **Print Screen**, la capture programmable permet l'automatisation, la sélection de régions spécifiques, et le traitement en temps réel.

2.1.1 Architecture de la Capture



2.1.2 Methodes de Capture

Il existe trois approches principales :

- **Capture complete** : Totalité de l'écran principal
- **Capture regionale** : Zone rectangulaire spécifique (x, y, largeur, hauteur)
- **Capture multi-ecrans** : Gestion de plusieurs moniteurs

2.2 Capture avec Pillow (PIL)

Pillow est la bibliothèque standard pour la manipulation d'images en Python.

Listing 2.1: Installation de Pillow

```
pip install Pillow
```

Listing 2.2: Capture complete avec Pillow

```
from PIL import ImageGrab
import datetime

def capturer_ecran_complet(dossier_sortie: str = ".") -> str:
    """
    Capture l'ecran complet et sauvegarde l'image.

    Returns:
        Chemin du fichier cree
    """
    timestamp =
        datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    nom_fichier = f"{dossier_sortie}/capture_{timestamp}.png"

    # Capture
    image = ImageGrab.grab()

    # Sauvegarde
    image.save(nom_fichier, "PNG")

    print(f"Capture sauvegardee : {nom_fichier}")
    print(f"Resolution : {image.size}")

    return nom_fichier

# Exemple d'utilisation
chemin = capturer_ecran_complet("./captures")
```

2.2.1 Capture Regionale

Listing 2.3: Capture d'une region specifique

```
from PIL import ImageGrab

def capturer_region(x: int, y: int, largeur: int, hauteur: int,
                    nom_fichier: str = "region.png") -> None:
    """
    Capture une region rectangulaire de l'ecran.

    Args:
        x, y: Coordonnees du coin superieur gauche
        largeur, hauteur: Dimensions de la region
    """
    # Definition de la boite (left, top, right, bottom)
    boite = (x, y, x + largeur, y + hauteur)

    # Capture
    image = ImageGrab.grab(bbox=boite)
```

```
# Sauvegarde
image.save(nom_fichier, "PNG")

print(f"Region capturee : {image.size}")

# Exemple : capturer le coin superieur gauche (500x500 pixels)
capturer_region(0, 0, 500, 500, "coin_superieur.png")
```

2.3 Capture Haute Performance avec MSS

MSS (Multiple Screen Shots) est optimise pour la vitesse et la capture multi-ecrans.

Listing 2.4: Installation de MSS

```
pip install mss
```

Listing 2.5: Capture rapide avec MSS

```
from mss import mss
from PIL import Image
import numpy as np

def capturer_avec_mss(moniteur: int = 1) -> np.ndarray:
    """
    Capture un ecran avec MSS (tres rapide).

    Args:
        moniteur: Numero du moniteur (1 = principal)

    Returns:
        Array numpy de l'image (RGB)
    """
    with mss() as sct:
        # Capture
        screenshot = sct.grab(sct.monitors[moniteur])

        # Conversion en array numpy
        image_array = np.array(screenshot)

        # MSS retourne BGRA, on convertit en RGB
        image_rgb = image_array[:, :, :3][:, :, ::-1]

    return image_rgb

# Exemple d'utilisation
image = capturer_avec_mss(1)
print(f"Shape : {image.shape}")
print(f"Type : {image.dtype}")

# Conversion en PIL pour sauvegarde
```



```
Image.fromarray(image).save("capture_mss.png")
```

2.3.1 Capture en Boucle (Monitoring)

Listing 2.6: Capture continue avec MSS

```
from mss import mss
import time

def monitorer_ecran(duree_secondes: int = 10, fps: int = 5) ->
    None:
    """
        Capture l'ecran en continu pendant une duree donnee.

        Args:
            duree_secondes: Duree totale de la capture
            fps: Images par seconde
    """
    intervalle = 1.0 / fps
    compteur = 0

    with mss() as sct:
        moniteur = sct.monitors[1]
        debut = time.time()

        while time.time() - debut < duree_secondes:
            t0 = time.time()

            # Capture
            screenshot = sct.grab(moniteur)

            # Sauvegarde
            nom = f"frame_{compteur:04d}.png"
            mss.tools.to_png(screenshot.rgb, screenshot.size,
                            output=nom)

            compteur += 1

            # Attente pour respecter le FPS
            temps_ecoule = time.time() - t0
            if temps_ecoule < intervalle:
                time.sleep(intervalle - temps_ecoule)

        print(f"Capture terminee : {compteur} frames en
              {duree_secondes}s")
        print(f"FPS moyen : {compteur / duree_secondes:.2f}")

    # Exemple : 10 secondes a 5 FPS
    monitorer_ecran(duree_secondes=10, fps=5)
```

2.4 Capture avec PyAutoGUI

PyAutoGUI offre des fonctionnalités supplémentaires comme la localisation d'images à l'écran.

Listing 2.7: Installation de PyAutoGUI

```
pip install pyautogui
```

Listing 2.8: Capture et localisation avec PyAutoGUI

```
import pyautogui

def capturer_et_localiser(image_reference: str) -> tuple:
    """
    Capture l'écran et localise une image de référence.

    Args:
        image_reference: Chemin vers l'image à localiser

    Returns:
        (x, y, largeur, hauteur) de l'image trouvée, ou None
    """
    # Capture
    screenshot = pyautogui.screenshot()
    screenshot.save("capture_pyautogui.png")

    # Localisation
    position = pyautogui.locateOnScreen(image_reference,
                                        confidence=0.8)

    if position:
        print(f"Image trouvée à : {position}")
        return position
    else:
        print("Image non trouvée")
        return None

# Exemple
position = capturer_et_localiser("icone_recherche.png")
```

2.5 Bonnes Pratiques

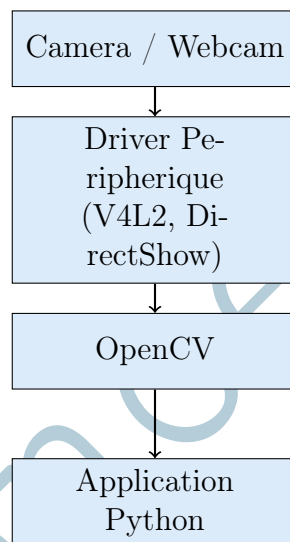
- **Performance** : Utilisez MSS pour les captures fréquentes (>10 FPS)
- **Multi-écrans** : Testez sur `sct.monitors` pour gérer plusieurs moniteurs
- **Format** : PNG pour la qualité, JPEG pour la compression
- **Timestamps** : Ajoutez toujours un horodatage aux noms de fichiers
- **Permissions** : Sur macOS, autorisez l'accès dans Préférences Système

Chapitre 3 Camera et Webcam

3.1 Principe Fondamental

L'acquisition d'images depuis une camera ou webcam permet la capture en temps reel du monde physique. C'est la base de la vision par ordinateur, de la reconnaissance faciale, de la videoconference et de nombreuses applications d'IA.

3.1.1 Architecture de Capture Video



3.2 Capture avec OpenCV

OpenCV (Open Source Computer Vision Library) est la reference pour la vision par ordinateur.

Listing 3.1: Installation de OpenCV

```
pip install opencv-python
```

3.2.1 Capture Simple d'une Frame

Listing 3.2: Capture d'une seule image depuis webcam

```
import cv2
import datetime

def capturer_photo_webcam(camera_id: int = 0,
                           dossier_sortie: str = ".") -> str:
    """
    Capture une photo depuis la webcam.
```

```

Args:
    camera_id: ID de la camera (0 = webcam par default)

Returns:
    Chemin du fichier cree
"""
# Ouverture de la camera
camera = cv2.VideoCapture(camera_id)

if not camera.isOpened():
    raise RuntimeError(f"Impossible d'ouvrir la camera
                       {camera_id}")

# Lecture d'une frame
ret, frame = camera.read()

if not ret:
    camera.release()
    raise RuntimeError("Impossible de lire la frame")

# Sauvegarde
timestamp =
    datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
nom_fichier = f"{dossier_sortie}/photo_{timestamp}.jpg"
cv2.imwrite(nom_fichier, frame)

# Liberation
camera.release()

print(f"Photo capturee : {nom_fichier}")
print(f"Resolution : {frame.shape[1]}x{frame.shape[0]}")

return nom_fichier

# Exemple
chemin = capturer_photo_webcam(0, "./photos")

```

3.2.2 Capture Video en Direct

Listing 3.3: Stream video en temps reel

```

import cv2

def stream_webcam(camera_id: int = 0) -> None:
    """
    Affiche le flux video de la webcam en temps reel.
    Appuyez sur 'q' pour quitter, 's' pour sauvegarder une
    frame.
    """
    camera = cv2.VideoCapture(camera_id)

```

```

if not camera.isOpened():
    print("Erreur : Impossible d'ouvrir la camera")
    return

print("Stream actif. Appuyez sur 'q' pour quitter, 's' pour
sauvegarder")
compteur_saves = 0

while True:
    ret, frame = camera.read()

    if not ret:
        print("Erreur de lecture")
        break

    # Affichage
    cv2.imshow('Webcam', frame)

    # Gestion clavier
    touche = cv2.waitKey(1) & 0xFF

    if touche == ord('q'):
        break
    elif touche == ord('s'):
        nom = f"capture_{compteur_saves:04d}.jpg"
        cv2.imwrite(nom, frame)
        print(f"Frame sauvegardee : {nom}")
        compteur_saves += 1

    # Nettoyage
    camera.release()
    cv2.destroyAllWindows()

# Exemple
stream_webcam(0)

```

3.2.3 Enregistrement Video

Listing 3.4: Enregistrement d'une video

```

import cv2

def enregistrer_video(camera_id: int = 0,
                      duree_secondes: int = 10,
                      fps: int = 30,
                      nom_fichier: str = "video.avi") -> None:
    """
    Enregistre une video depuis la webcam.

    Args:
    """

```

```
    duree_secondes: Duree de l'enregistrement
    fps: Images par seconde
"""
camera = cv2.VideoCapture(camera_id)

if not camera.isOpened():
    print("Erreur : Impossible d'ouvrir la camera")
    return

# Configuration
largeur = int(camera.get(cv2.CAP_PROP_FRAME_WIDTH))
hauteur = int(camera.get(cv2.CAP_PROP_FRAME_HEIGHT))

# Codec et writer
fourcc = cv2.VideoWriter_fourcc(*'XVID')
writer = cv2.VideoWriter(nom_fichier, fourcc, fps,
                        (largeur, hauteur))

print(f"Enregistrement en cours ({duree_secondes}s)...")

frames_total = duree_secondes * fps
frames_capturees = 0

while frames_capturees < frames_total:
    ret, frame = camera.read()

    if not ret:
        break

    writer.write(frame)
    frames_capturees += 1

# Affichage progression
if frames_capturees % fps == 0:
    print(f"  {frames_capturees // fps}s /
          {duree_secondes}s")

# Nettoyage
camera.release()
writer.release()

print(f"Video enregistree : {nom_fichier}")
print(f"Frames capturees : {frames_capturees}")

# Exemple : 10 secondes a 30 FPS
enregistrer_video(0, duree_secondes=10, fps=30,
                  nom_fichier="ma_video.avi")
```

3.3 Configuration Avancee

3.3.1 Parametres de la Camera

Listing 3.5: Configuration des parametres camera

```
import cv2

def configurer_camera(camera_id: int = 0) -> cv2.VideoCapture:
    """
    Configure les parametres de la camera.
    """
    camera = cv2.VideoCapture(camera_id)

    # Resolution
    camera.set(cv2.CAP_PROP_FRAME_WIDTH, 1920)
    camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 1080)

    # FPS
    camera.set(cv2.CAP_PROP_FPS, 30)

    # Exposition
    camera.set(cv2.CAP_PROP_AUTO_EXPOSURE, 0.25) # Mode manuel
    camera.set(cv2.CAP_PROP_EXPOSURE, -6)

    # Verification
    largeur = camera.get(cv2.CAP_PROP_FRAME_WIDTH)
    hauteur = camera.get(cv2.CAP_PROP_FRAME_HEIGHT)
    fps = camera.get(cv2.CAP_PROP_FPS)

    print(f"Configuration camera :")
    print(f"  Resolution : {int(largeur)}x{int(hauteur)}")
    print(f"  FPS : {fps}")

    return camera

# Exemple
cam = configurer_camera(0)
```

3.4 Detection de Visages en Temps Reel

Listing 3.6: Detection de visages avec Haar Cascades

```
import cv2

def detecter_visages_webcam(camera_id: int = 0) -> None:
    """
    Detecte les visages en temps reel et les encadre.
    """
    # Chargement du classifieur Haar
    cascade_visage = cv2.CascadeClassifier(
```

```

cv2.data.harcascades +
    'haarcascade_frontalface_default.xml'
)

camera = cv2.VideoCapture(camera_id)

print("Detection active. Appuyez sur 'q' pour quitter")

while True:
    ret, frame = camera.read()

    if not ret:
        break

    # Conversion en niveaux de gris (requis pour Haar)
    gris = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detection
    visages = cascade_visage.detectMultiScale(
        gris,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(30, 30)
    )

    # Dessin des rectangles
    for (x, y, w, h) in visages:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255,
            0), 2)

    # Affichage nombre de visages
    cv2.putText(frame, f"Visages : {len(visages)}",
        (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
        1, (0, 255, 0), 2)

    cv2.imshow('Detection Visages', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

    camera.release()
    cv2.destroyAllWindows()

# Exemple
detection_visages_webcam(0)

```

3.5 Bonnes Pratiques

- **Liberation** : Toujours appeler `camera.release()` et `cv2.destroyAllWindows()`

- **Erreurs** : Verifiez `camera.isOpened()` et `ret` avant d'utiliser une frame
- **Resolution** : Adaptez la resolution a votre use case (720p suffisent souvent)
- **FPS** : 30 FPS pour video fluide, 10-15 FPS pour traitement temps reel
- **Codec** : XVID pour AVI, H264 pour MP4, MJPEG pour compatibilite maximale

Licence IRM

Chapitre 4 Scanner

4.1 Principe Fondamental

Un scanner numérise des documents ou photographies physiques en images numériques haute résolution. Contrairement aux caméras, les scanners offrent une qualité optique supérieure et une géométrie plane parfaite, idéale pour l'archivage et l'OCR.

4.1.1 Types de Scanners

- **Scanner à plat** : Document pose sur une vitre, scan ligne par ligne
- **Scanner à défilement** : Document traverse le scanner (ADF - Automatic Document Feeder)
- **Scanner portable** : Compact, USB, idéal pour le terrain
- **Scanner 3D** : Capture la géométrie tridimensionnelle

4.2 Protocoles de Communication

4.2.1 TWAIN (Windows/macOS)

TWAIN est le standard industriel pour les scanners sur Windows et macOS.

Listing 4.1: Installation de python-twain

```
pip install python-twain
```

Listing 4.2: Scan avec TWAIN (Windows)

```
import twain

def scanner_avec_twain(nom_fichier: str = "scan.png") -> None:
    """
    Lance une acquisition depuis un scanner TWAIN.
    Une interface graphique s'ouvre pour sélectionner le
    scanner.
    """
    # Initialisation du gestionnaire TWAIN
    sm = twain.SourceManager(0)

    # Sélection du scanner (interface graphique)
    scanner = sm.OpenSource()

    if not scanner:
        print("Aucun scanner sélectionné")
        return
```

```

# Configuration
scanner.SetCapability(twain.ICAP_PIXELTYPE, twain.TWPT_RGB)
scanner.SetCapability(twain.ICAP_XRESOLUTION, 300) # DPI
scanner.SetCapability(twain.ICAP_YRESOLUTION, 300)

# Acquisition
scanner.RequestAcquire(0, 0) # Mode non-modal

# Recuperation de l'image
image = scanner.XferImageNatively()[0]

# Sauvegarde
image.save(nom_fichier)

print(f"Scan sauvegarde : {nom_fichier}")
print(f"Resolution : {image.size}")

# Exemple
scanner_avec_twain("document_scanne.png")

```

4.2.2 SANE (Linux)

SANE (Scanner Access Now Easy) est le standard sur Linux.

Listing 4.3: Installation de python-sane

```

# Sur Ubuntu/Debian
sudo apt-get install sane-utils libsane-dev
pip install python-sane

```

Listing 4.4: Scan avec SANE (Linux)

```

import sane
from PIL import Image

def lister_scanners() -> list:
    """
    Liste tous les scanners detectes par SANE.
    """
    sane.init()
    scanners = sane.get_devices()

    print(f"Scanners detectes : {len(scanners)}")
    for i, (nom, fabricant, modele, type_) in enumerate(scanners):
        print(f" [{i}] {fabricant} {modele} ({nom})")

    return scanners

def scanner_avec_sane(scanner_id: int = 0,
                      resolution: int = 300,

```

```

        nom_fichier: str = "scan.png") -> None:
    """
    Numerise un document avec SANE.

    Args:
        scanner_id: Index du scanner (voir lister_scanners)
        resolution: Resolution en DPI
    """
    sane.init()
    scanners = sane.get_devices()

    if scanner_id >= len(scanners):
        print(f"Scanner {scanner_id} introuvable")
        return

    # Ouverture du scanner
    scanner = sane.open(scanners[scanner_id][0])

    # Configuration
    scanner.mode = 'color'
    scanner.resolution = resolution

    # Scan
    scanner.start()
    image_array = scanner.arr_snap()

    # Conversion en PIL Image
    image = Image.fromarray(image_array)

    # Sauvegarde
    image.save(nom_fichier)

    # Fermeture
    scanner.close()

    print(f"Scan sauvegarde : {nom_fichier}")
    print(f"Resolution : {image.size}")

# Exemple
lister_scanners()
scanner_avec_sane(0, resolution=300, nom_fichier="document.png")

```

4.3 Scan Multi-Pages (ADF)

Pour les scanners equips d'un chargeur automatique de documents (ADF) :

Listing 4.5: Scan multi-pages avec SANE

```

import sane
from PIL import Image
import os

```

```

def scanner_multipages(dossier_sortie: str = "./scans",
                      resolution: int = 300) -> list:
    """
    Numerise plusieurs pages depuis l'ADF.

    Returns:
        Liste des chemins des fichiers crees
    """
    os.makedirs(dossier_sortie, exist_ok=True)

    sane.init()
    scanners = sane.get_devices()
    scanner = sane.open(scanners[0][0])

    # Configuration
    scanner.mode = 'color'
    scanner.resolution = resolution
    scanner.source = 'ADF' # Automatic Document Feeder

    fichiers = []
    page = 1

    print("Scan multi-pages en cours...")

    while True:
        try:
            scanner.start()
            image_array = scanner.arr_snap()

            # Sauvegarde
            nom = f"{dossier_sortie}/page_{page:03d}.png"
            Image.fromarray(image_array).save(nom)

            fichiers.append(nom)
            print(f"  Page {page} scannee")
            page += 1

        except Exception as e:
            # Fin du chargeur
            print(f"Scan termine : {page - 1} pages")
            break

    scanner.close()

    return fichiers

# Exemple
fichiers = scanner_multipages("./mes_scans", resolution=300)

```

4.4 Post-Traitement des Scans

4.4.1 Redressement (Deskew)

Listing 4.6: Redressement automatique d'un scan

```
import cv2
import numpy as np
from PIL import Image

def redresser_scan(chemin_image: str,
                  chemin_sortie: str = None) -> None:
    """
    Redresse un scan incline automatiquement.
    """
    # Lecture
    image = cv2.imread(chemin_image)
    gris = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Detection des bords
    edges = cv2.Canny(gris, 50, 150, apertureSize=3)

    # Detection de lignes avec transformee de Hough
    lignes = cv2.HoughLines(edges, 1, np.pi / 180, 200)

    if lignes is None:
        print("Aucune ligne detectee, pas de redressement")
        return

    # Calcul de l'angle median
    angles = []
    for ligne in lignes:
        rho, theta = ligne[0]
        angle = np.degrees(theta) - 90
        if -45 < angle < 45:
            angles.append(angle)

    if not angles:
        print("Pas d'angle de correction trouve")
        return

    angle_median = np.median(angles)

    # Rotation
    (h, w) = image.shape[:2]
    centre = (w // 2, h // 2)
    matrice = cv2.getRotationMatrix2D(centre, angle_median, 1.0)
    image_redresse = cv2.warpAffine(image, matrice, (w, h),
                                    flags=cv2.INTER_CUBIC,
                                    borderMode=cv2.BORDER_REPLICATE)

    # Sauvegarde
```

```
if chemin_sortie is None:
    chemin_sortie = chemin_image.replace(".png",
        "_redresse.png")

cv2.imwrite(chemin_sortie, image_redresse)

print(f"Scan redresse de {angle_median:.2f} degrees")
print(f"Fichier : {chemin_sortie}")

# Exemple
redresser_scan("scan_incline.png", "scan_droit.png")
```

4.4.2 Suppression du Bruit

Listing 4.7: Nettoyage d'un scan bruité

```
import cv2

def nettoyer_scan(chemin_image: str,
                  chemin_sortie: str = None) -> None:
    """
    Supprime le bruit et ameliore le contraste.
    """
    # Lecture
    image = cv2.imread(chemin_image)

    # Debruitage
    image_propre = cv2.fastNlMeansDenoisingColored(
        image,
        None,
        h=10,
        hColor=10,
        templateWindowSize=7,
        searchWindowSize=21
    )

    # Amelioration du contraste
    lab = cv2.cvtColor(image_propre, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)

    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    l = clahe.apply(l)

    lab = cv2.merge([l, a, b])
    image_finale = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)

    # Sauvegarde
    if chemin_sortie is None:
        chemin_sortie = chemin_image.replace(".png",
            "_nettoye.png")
```

```
cv2.imwrite(chemin_sortie, image_finale)

print(f"Scan nettoye : {chemin_sortie}")

# Exemple
nettoyer_scan("scan_bruite.png", "scan_propre.png")
```

4.5 Bonnes Pratiques

- **Resolution** : 300 DPI pour documents, 600+ DPI pour photos
- **Format** : TIFF pour archivage (sans perte), PNG pour qualite, PDF pour multi-pages
- **Couleur** : RGB pour photos, niveaux de gris pour documents, noir/blanc pour texte
- **Calibration** : Nettoyez regulierement la vitre du scanner
- **Batch** : Utilisez l'ADF pour numeriser plusieurs documents d'un coup

Chapitre 5 Téléchargement d'Images

5.1 Principe Fondamental

Le téléchargement d'images via HTTP/HTTPS ou APIs permet d'obtenir des données visuelles depuis le web. C'est essentiel pour construire des datasets d'apprentissage automatique, agréger du contenu visuel, ou accéder à des bases d'images publiques.

5.2 Téléchargement Simple avec Requests

Listing 5.1: Installation de Requests

```
pip install requests Pillow
```

Listing 5.2: Téléchargement basique d'une image

```
import requests
from PIL import Image
from io import BytesIO

def telecharger_image(url: str, nom_fichier: str = None) -> str:
    """
    Télécharge une image depuis une URL.

    Returns:
        Chemin du fichier sauvegarde
    """
    # Requete HTTP
    response = requests.get(url, timeout=10)
    response.raise_for_status()

    # Verification du type MIME
    content_type = response.headers.get('Content-Type', '')
    if 'image' not in content_type:
        raise ValueError(f"L'URL ne pointe pas vers une image : {content_type}")

    # Ouverture avec Pillow
    image = Image.open(BytesIO(response.content))

    # Nom de fichier automatique si non fourni
    if nom_fichier is None:
        extension = image.format.lower()
        nom_fichier = f"image_{hash(url) % 100000}.{extension}"

    # Sauvegarde
```

```

image.save(nom_fichier)

print(f"Image telechargee : {nom_fichier}")
print(f"  Resolution : {image.size}")
print(f"  Format : {image.format}")
print(f"  Taille : {len(response.content) / 1024:.2f} Ko")

return nom_fichier

# Exemple
url = "https://picsum.photos/800/600"
chemin = telecharger_image(url, "ma_photo.jpg")

```

5.2.1 Telechargement Multiple

Listing 5.3: Telechargement de plusieurs images

```

import requests
from PIL import Image
from io import BytesIO
import os

def telecharger_images(urls: list, dossier_sortie: str =
    "./images") -> list:
    """
    Telecharge plusieurs images.

    Returns:
        Liste des chemins des fichiers crees
    """
    os.makedirs(dossier_sortie, exist_ok=True)
    fichiers = []

    for i, url in enumerate(urls):
        try:
            response = requests.get(url, timeout=10)
            response.raise_for_status()

            image = Image.open(BytesIO(response.content))

            # Nom de fichier
            extension = image.format.lower()
            nom = f"{dossier_sortie}/image_{i:04d}.{extension}"

            image.save(nom)
            fichiers.append(nom)

            print(f"[{i+1}/{len(urls)}] {nom} ({image.size})")

        except Exception as e:
            print(f"[{i+1}/{len(urls)}] ERREUR : {e}")

```

```
print(f"\nTelecharges : {len(fichiers)}/{len(urls)}")
return fichiers

# Exemple
urls = [
    "https://picsum.photos/800/600?random=1",
    "https://picsum.photos/800/600?random=2",
    "https://picsum.photos/800/600?random=3"
]
fichiers = telecharger_images(urls, "./mes_images")
```

5.3 Telechargement Asynchrone

Pour telecharger des centaines d'images rapidement, utilisez aiohttp :

Listing 5.4: Installation des dependances asynchrones

```
pip install aiohttp aiofiles
```

Listing 5.5: Telechargement asynchrone haute performance

```
import aiohttp
import asyncio
import aiofiles
import os

async def telecharger_image_async(session:
    aiohttp.ClientSession,
                                url: str,
                                nom_fichier: str) -> bool:
    """
    Telecharge une image de maniere asynchrone.
    """
    try:
        async with session.get(url, timeout=10) as response:
            if response.status != 200:
                return False

            contenu = await response.read()

            async with aiofiles.open(nom_fichier, 'wb') as f:
                await f.write(contenu)

            return True
    except Exception as e:
        print(f"Erreur {url}: {e}")
        return False

async def telecharger_images_async(urls: list,
                                    dossier_sortie: str =
                                        "./images") -> int:
```

```

"""
Telecharge plusieurs images en parallele.

Returns:
    Nombre d'images telechargees avec succes
"""
os.makedirs(dossier_sortie, exist_ok=True)

async with aiohttp.ClientSession() as session:
    taches = []

    for i, url in enumerate(urls):
        nom = f"{dossier_sortie}/image_{i:04d}.jpg"
        tache = telecharger_image_async(session, url, nom)
        taches.append(tache)

    resultats = await asyncio.gather(*taches)

    succes = sum(resultats)
    print(f"Telecharges : {succes}/{len(urls)}")

    return succes

# Exemple
urls = [f"https://picsum.photos/800/600?random={i}" for i in
        range(50)]
asyncio.run(telecharger_images_async(urls, "./dataset"))

```

5.4 APIs d'Images

5.4.1 Unsplash API

Unsplash offre une API gratuite pour acceder a des millions de photos haute qualite.

Listing 5.6: Recherche d'images avec Unsplash API

```

import requests

def rechercher_unsplash(query: str,
                        nb_images: int = 10,
                        access_key: str = "VOTRE_CLE") -> list:
    """
    Recherche des images sur Unsplash.

    Returns:
        Liste d'URLs d'images
    """
    url = "https://api.unsplash.com/search/photos"

    headers = {
        "Authorization": f"Client-ID {access_key}"
    }

```

```

}

params = {
    "query": query,
    "per_page": nb_images,
    "orientation": "landscape"
}

response = requests.get(url, headers=headers, params=params)
response.raise_for_status()

data = response.json()

# Extraction des URLs
urls = []
for photo in data['results']:
    url_image = photo['urls']['regular']
    urls.append(url_image)
    print(f"  {photo['description']} -
          {photo['width']}x{photo['height']}")

return urls

# Exemple
urls = rechercher_unsplash("mountain", nb_images=10,
    access_key="VOTRE_CLE")

```

5.4.2 Pexels API

Listing 5.7: Téléchargement depuis Pexels

```

import requests

def telecharger_pexels(query: str,
                        nb_images: int = 10,
                        api_key: str = "VOTRE_CLE") -> list:
    """
    Télécharge des images depuis Pexels.
    """
    url = "https://api.pexels.com/v1/search"

    headers = {
        "Authorization": api_key
    }

    params = {
        "query": query,
        "per_page": nb_images
    }

    response = requests.get(url, headers=headers, params=params)

```

```
response.raise_for_status()

data = response.json()

urls = []
for photo in data['photos']:
    url_image = photo['src']['large']
    urls.append(url_image)

return urls

# Exemple
urls = telecharger_pexels("ocean", nb_images=10,
    api_key="VOTRE_CLE")
```

5.5 Scraping d'Images

Pour extraire des images depuis des pages web :

Listing 5.8: Scraping d'images avec BeautifulSoup

```
import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin

def extraire_images_page(url_page: str) -> list:
    """
    Extrait toutes les URLs d'images depuis une page web.

    Returns:
        Liste d'URLs absolues
    """
    response = requests.get(url_page, timeout=10)
    response.raise_for_status()

    soup = BeautifulSoup(response.content, 'html.parser')

    # Trouver toutes les balises <img>
    images = soup.find_all('img')

    urls = []
    for img in images:
        src = img.get('src')
        if src:
            # Conversion en URL absolue
            url_absolue = urljoin(url_page, src)
            urls.append(url_absolue)

    print(f"Images trouvees : {len(urls)}")

    return urls
```

```
# Exemple
urls = extraire_images_page("https://example.com")
```

5.6 Bonnes Pratiques

- **Respect** : Verifiez les CGU et robots.txt des sites
- **Rate Limiting** : Ajoutez des delais entre les requetes (`time.sleep`)
- **Erreurs** : Gerez les timeouts, 404, et erreurs reseau
- **Format** : Verifiez le Content-Type avant de sauvegarder
- **Deduplication** : Utilisez des hashes pour eviter les doublons

Chapitre 6 Generation d'Images par IA

6.1 Principe Fondamental

La generation d'images par IA consiste a creer synthetiquement des images a partir de descriptions textuelles (text-to-image) ou d'autres images (image-to-image). Les modeles de diffusion comme Stable Diffusion, DALL-E et Midjourney ont revolutionne ce domaine.

6.1.1 Modeles Principaux

Modele	Type	Acces	Qualite
Stable Diffusion	Open-source	Local + API	Tres bonne
DALL-E 3	Proprietaire	API OpenAI	Excellente
Midjourney	Proprietaire	API Discord	Excellente
Imagen	Proprietaire	API Google	Excellente

6.2 Stable Diffusion (Local)

Stable Diffusion peut tourner localement sur un GPU.

Listing 6.1: Installation de Diffusers

```
pip install diffusers transformers accelerate safetensors
```

Listing 6.2: Generation avec Stable Diffusion

```
from diffusers import StableDiffusionPipeline
import torch

def generer_image_sd(prompt: str,
                    modele: str =
                        "stabilityai/stable-diffusion-2-1",
                    nom_fichier: str = "generated.png") -> str:
    """
    Genere une image avec Stable Diffusion.

    Args:
        prompt: Description textuelle de l'image
        modele: ID du modele Hugging Face

    Returns:
        Chemin du fichier cree
    """
    # Chargement du modele
    pipe = StableDiffusionPipeline.from_pretrained(
        modele,
```



```

        torch_dtype=torch.float16
    )

    # GPU si disponible
    if torch.cuda.is_available():
        pipe = pipe.to("cuda")
        print("GPU detecte : generation acceleree")
    else:
        print("CPU : generation lente")

    # Generation
    print(f"Generation en cours : '{prompt}'...")
    image = pipe(prompt).images[0]

    # Sauvegarde
    image.save(nom_fichier)

    print(f"Image generee : {nom_fichier}")
    print(f"Resolution : {image.size}")

    return nom_fichier

# Exemple
prompt = "A serene mountain landscape at sunset,
photorealistic, 4k"
chemin = generer_image_sd(prompt, nom_fichier="montagne.png")

```

6.2.1 Parametres Avances

Listing 6.3: Generation avec parametres personnalisés

```

from diffusers import StableDiffusionPipeline
import torch

def generer_avance(prompt: str,
                    negative_prompt: str = "",
                    steps: int = 50,
                    guidance_scale: float = 7.5,
                    seed: int = None,
                    nom_fichier: str = "image.png") -> str:
    """
    Generation avec controle total des parametres.

    Args:
        negative_prompt: Ce qu'on NE veut PAS dans l'image
        steps: Nombre d'etapes de debruitage (plus = meilleur)
        guidance_scale: Force de guidage (7-15 recommande)
        seed: Graine aleatoire pour reproductibilite
    """
    pipe = StableDiffusionPipeline.from_pretrained(
        "stabilityai/stable-diffusion-2-1",

```

```

        torch_dtype=torch.float16
    ).to("cuda")

    # Générateur pour seed
    generator = None
    if seed is not None:
        generator = torch.Generator("cuda").manual_seed(seed)

    # Génération
    image = pipe(
        prompt=prompt,
        negative_prompt=negative_prompt,
        num_inference_steps=steps,
        guidance_scale=guidance_scale,
        generator=generator
    ).images[0]

    image.save(nom_fichier)

    print(f"Genere : {nom_fichier}")
    return nom_fichier

# Exemple
prompt = "beautiful portrait of a woman, studio lighting, professional"
negative = "blurry, low quality, distorted, ugly"
generer_avance(prompt, negative_prompt=negative, steps=50,
               seed=42)

```

6.3 DALL-E 3 via OpenAI API

Listing 6.4: Installation du SDK OpenAI

```
pip install openai
```

Listing 6.5: Génération avec DALL-E 3

```

from openai import OpenAI
import requests
from PIL import Image
from io import BytesIO

def generer_dalle3(prompt: str,
                  api_key: str,
                  size: str = "1024x1024",
                  quality: str = "standard",
                  nom_fichier: str = "dalle3.png") -> str:
    """
    Genere une image avec DALL-E 3.

    Args:
    """

```

```

        size: "1024x1024", "1024x1792", "1792x1024"
        quality: "standard" ou "hd"
    """
    client = OpenAI(api_key=api_key)

    # Generation
    print(f"Generation DALL-E 3 : '{prompt}'...")
    response = client.images.generate(
        model="dall-e-3",
        prompt=prompt,
        size=size,
        quality=quality,
        n=1
    )

    # Telechargement
    url_image = response.data[0].url
    response_img = requests.get(url_image)

    image = Image.open(BytesIO(response_img.content))
    image.save(nom_fichier)

    print(f"Image generee : {nom_fichier}")
    print(f"  Prompt revise :
          {response.data[0].revised_prompt}")

    return nom_fichier

# Exemple
prompt = "A futuristic city at night with neon lights"
generer_dalle3(prompt, api_key="VOTRE_CLE", quality="hd")

```

6.4 Image-to-Image

Modifier une image existante avec l'IA :

Listing 6.6: Transformation image-to-image

```

from diffusers import StableDiffusionImg2ImgPipeline
from PIL import Image
import torch

def transformer_image(chemin_image: str,
                      prompt: str,
                      strength: float = 0.75,
                      nom_sortie: str = "transformed.png") ->
    str:
    """
    Transforme une image existante selon un prompt.

    Args:

```

```

        strength: Force de transformation (0.0 = identique, 1.0
            = nouvelle)
    """
    # Chargement du modele
    pipe = StableDiffusionImg2ImgPipeline.from_pretrained(
        "stabilityai/stable-diffusion-2-1",
        torch_dtype=torch.float16
    ).to("cuda")

    # Chargement de l'image source
    image_source = Image.open(chemin_image).convert("RGB")
    image_source = image_source.resize((512, 512))

    # Transformation
    image_resultat = pipe(
        prompt=prompt,
        image=image_source,
        strength=strength,
        guidance_scale=7.5
    ).images[0]

    image_resultat.save(nom_sortie)

    print(f"Image transformee : {nom_sortie}")
    return nom_sortie

# Exemple
transformer_image(
    "photo_originale.jpg",
    "same scene but in winter with snow",
    strength=0.7,
    nom_sortie="hiver.png"
)

```

6.5 Inpainting

Remplir ou modifier des regions specifiques d'une image :

Listing 6.7: Inpainting avec masque

```

from diffusers import StableDiffusionInpaintPipeline
from PIL import Image
import torch

def inpainting(chemin_image: str,
               chemin_masque: str,
               prompt: str,
               nom_sortie: str = "inpainted.png") -> str:
    """
    Remplit les zones masquees selon le prompt.
    """

```

```
Le masque doit etre noir (0) pour garder, blanc (255) pour
remplacer.
"""
pipe = StableDiffusionInpaintPipeline.from_pretrained(
    "stabilityai/stable-diffusion-2-inpainting",
    torch_dtype=torch.float16
).to("cuda")

# Chargement
image =
    Image.open(chemin_image).convert("RGB").resize((512, 512))
masque =
    Image.open(chemin_masque).convert("L").resize((512, 512))

# Inpainting
resultat = pipe(
    prompt=prompt,
    image=image,
    mask_image=masque
).images[0]

resultat.save(nom_sortie)

print(f"Inpainting termine : {nom_sortie}")
return nom_sortie

# Exemple
inpainting(
    "photo.jpg",
    "masque.png",
    "a red sports car",
    "modified.png"
)
```

6.6 Bonnes Pratiques

- **GPU** : Un GPU avec 8+ Go de VRAM est recommande pour Stable Diffusion
- **Prompts** : Soyez precis, ajoutez des qualificatifs (photorealistic, 4k, detailed)
- **Negative Prompts** : Specifiez ce que vous NE voulez PAS
- **Seeds** : Sauvegardez les seeds pour reproduire les bonnes generations
- **Iteration** : Generez plusieurs variantes et selectionnez la meilleure

Chapitre 7 Extraction de Frames Video

7.1 Principe Fondamental

L'extraction de frames consiste a decomposer une video en une sequence d'images fixes. C'est essentiel pour l'analyse temporelle, la detection d'evenements, la creation de time-lapses, ou la construction de datasets d'apprentissage.

7.2 Extraction avec OpenCV

Listing 7.1: Extraction de toutes les frames

```
import cv2
import os

def extraire_toutes_frames(chemin_video: str,
                           dossier_sortie: str = "./frames") ->
                           int:
    """
    Extrait toutes les frames d'une video.

    Returns:
        Nombre de frames extraites
    """
    os.makedirs(dossier_sortie, exist_ok=True)

    # Ouverture video
    video = cv2.VideoCapture(chemin_video)

    if not video.isOpened():
        raise RuntimeError(f"Impossible d'ouvrir
                           {chemin_video}")

    # Informations video
    fps = video.get(cv2.CAP_PROP_FPS)
    total_frames = int(video.get(cv2.CAP_PROP_FRAME_COUNT))
    duree = total_frames / fps

    print(f"Video : {total_frames} frames, {fps} FPS,
          {duree:.2f}s")

    # Extraction
    compteur = 0

    while True:
```

```

ret, frame = video.read()

if not ret:
    break

# Sauvegarde
nom = f"{dossier_sortie}/frame_{compteur:05d}.jpg"
cv2.imwrite(nom, frame)

compteur += 1

if compteur % 100 == 0:
    print(f"  Extrait : {compteur}/{total_frames}")

video.release()

print(f"Extraction terminee : {compteur} frames")
return compteur

# Exemple
nb_frames = extraire_toutes_frames("ma_video.mp4", "./frames")

```

7.2.1 Extraction Selective

Listing 7.2: Extraction d'une frame par seconde

```

import cv2
import os

def extraire_fps_fixe(chemin_video: str,
                      fps_sortie: int = 1,
                      dossier_sortie: str = "./frames") -> int:
    """
    Extrait X frames par seconde.

    Args:
        fps_sortie: Nombre de frames par seconde a extraire
    """
    os.makedirs(dossier_sortie, exist_ok=True)

    video = cv2.VideoCapture(chemin_video)
    fps_video = video.get(cv2.CAP_PROP_FPS)

    # Calculer l'intervalles
    intervalle = int(fps_video / fps_sortie)

    print(f"Video : {fps_video} FPS")
    print(f"Extraction : 1 frame toutes les {intervalle} frames")

    compteur_frame = 0

```

```

compteur_sauve = 0

while True:
    ret, frame = video.read()

    if not ret:
        break

    # Extraction selective
    if compteur_frame % intervalle == 0:
        nom =
            f"{dossier_sortie}/frame_{compteur_sauve:05d}.jpg"
        cv2.imwrite(nom, frame)
        compteur_sauve += 1

    compteur_frame += 1

video.release()

print(f"Frames extraites : {compteur_sauve}")
return compteur_sauve

# Exemple : 1 frame par seconde
extraire_fps_fixe("video.mp4", fps_sortie=1,
    dossier_sortie="./timelapse")

```

7.2.2 Extraction par Intervalle Temporel

Listing 7.3: Extraction entre deux timestamps

```

import cv2
import os

def extraire_intervalle(chemin_video: str,
                        debut_sec: float,
                        fin_sec: float,
                        dossier_sortie: str = "./frames") ->
    int:
    """
    Extrait les frames entre deux timestamps.

    Args:
        debut_sec, fin_sec: Timestamps en secondes
    """
    os.makedirs(dossier_sortie, exist_ok=True)

    video = cv2.VideoCapture(chemin_video)
    fps = video.get(cv2.CAP_PROP_FPS)

    # Conversion en numero de frame
    frame_debut = int(debut_sec * fps)

```



```

frame_fin = int(fin_sec * fps)

# Positionnement
video.set(cv2.CAP_PROP_POS_FRAMES, frame_debut)

print(f"Extraction de {debut_sec}s a {fin_sec}s")
print(f"Frames {frame_debut} a {frame_fin}")

compteur = 0

for i in range(frame_debut, frame_fin):
    ret, frame = video.read()

    if not ret:
        break

    nom = f"{dossier_sortie}/frame_{compteur:05d}.jpg"
    cv2.imwrite(nom, frame)
    compteur += 1

video.release()

print(f"Frames extraites : {compteur}")
return compteur

# Exemple : de 10s a 20s
extraire_intervalle("video.mp4", debut_sec=10, fin_sec=20)

```

7.3 Detection de Changements de Scene

Extraire uniquement les frames ou la scene change :

Listing 7.4: Detection de changements de scene

```

import cv2
import numpy as np
import os

def extraire_changements_scene(chemin_video: str,
                               seuil: float = 30.0,
                               dossier_sortie: str =
                                   "./scenes") -> int:
    """
    Extrait les frames ou la scene change significativement.

    Args:
        seuil: Seuil de difference (plus bas = plus sensible)
    """
    os.makedirs(dossier_sortie, exist_ok=True)

    video = cv2.VideoCapture(chemin_video)

```

```

ret, frame_precedente = video.read()
if not ret:
    return 0

# Conversion en niveaux de gris
gris_prec = cv2.cvtColor(frame_precedente,
    cv2.COLOR_BGR2GRAY)

compteur_scenes = 0
compteur_frame = 0

# Sauvegarde premiere frame
cv2.imwrite(f"{dossier_sortie}/scene_{compteur_scenes:04d}.jpg",
    frame_precedente)
compteur_scenes += 1

print("Detection de scenes en cours...")

while True:
    ret, frame = video.read()

    if not ret:
        break

    compteur_frame += 1

    # Conversion
    gris = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Calcul de la difference
    difference = cv2.absdiff(gris_prec, gris)
    score = np.mean(difference)

    # Detection changement
    if score > seuil:
        nom =
            f"{dossier_sortie}/scene_{compteur_scenes:04d}.jpg"
        cv2.imwrite(nom, frame)
        print(f"  Scene {compteur_scenes} detectee (frame
            {compteur_frame}, "
            f"  score={score:.2f})")
        compteur_scenes += 1

    gris_prec = gris

video.release()

print(f"Scenes detectees : {compteur_scenes}")
return compteur_scenes

```

```
# Exemple
extraire_changements_scene("film.mp4", seuil=30.0,
    dossier_sortie="./scenes")
```

7.4 Creation de Mosaïque

Assembler plusieurs frames en une seule image :

Listing 7.5: Mosaïque de frames

```
import cv2
import numpy as np
from PIL import Image

def creer_mosaïque(chemin_video: str,
                   nb_frames: int = 16,
                   grille: tuple = (4, 4),
                   nom_sortie: str = "mosaïque.jpg") -> str:
    """
    Cree une mosaïque de frames regulierement espacees.

    Args:
        nb_frames: Nombre total de frames a extraire
        grille: (lignes, colonnes)
    """
    video = cv2.VideoCapture(chemin_video)
    total_frames = int(video.get(cv2.CAP_PROP_FRAME_COUNT))

    # Calcul des positions
    intervalle = total_frames // nb_frames

    frames = []

    for i in range(nb_frames):
        position = i * intervalle
        video.set(cv2.CAP_PROP_POS_FRAMES, position)

        ret, frame = video.read()
        if ret:
            # Redimensionnement
            frame_petit = cv2.resize(frame, (320, 180))
            frames.append(frame_petit)

    video.release()

    # Assemblage
    lignes, colonnes = grille
    mosaïque_lignes = []

    for i in range(lignes):
        debut = i * colonnes
```

```
    fin = debut + colonnes
    ligne = np.hstack(frames[debut:fin])
    mosaique_lignes.append(ligne)

mosaique = np.vstack(mosaique_lignes)

# Sauvegarde
cv2.imwrite(nom_sortie, mosaique)

print(f"Mosaique creee : {nom_sortie}")
print(f"  Dimensions :
      {mosaique.shape[1]}x{mosaique.shape[0]}")

return nom_sortie

# Exemple : grille 4x4 de 16 frames
creer_mosaique("video.mp4", nb_frames=16, grille=(4, 4))
```

7.5 Bonnes Pratiques

- **Format** : JPEG pour economies d'espace, PNG si qualite critique
- **Nomenclature** : Utilisez des noms avec zero-padding (frame_00001.jpg)
- **Metadata** : Sauvegardez FPS, resolution, codec dans un fichier JSON
- **Espace disque** : 1 frame 1080p $\approx$ 200 Ko (JPEG), surveillez l'espace
- **Performance** : Utilisez `cv2.CAP_PROP_POS_FRAMES` pour sauter des frames

Chapitre 8 Conclusion

8.1 Tableau Comparatif des Mecanismes

Mecanisme	Use Cases	Outils Python	Qualite
Capture ecran	Tests UI, monitoring, bots	Pillow, mss, pyautogui	Parfaite
Camera/Webcam	Vision par ordinateur, securite	OpenCV, picamera	Variable
Scanner	Archivage, OCR, preservation	python-twain, sane	Excellente
Telechargement	Datasets ML, scraping visuel	requests, aiohttp	Variable
Generation IA	Creation contenu, augmentation	diffusers, openai	Synthetique
Extraction video	Analyse temporelle, timelapse	OpenCV, ffmpeg	Variable

8.2 Bonnes Pratiques Generales

1. Choisir le bon mecanisme selon la source

- Ecran ? → Capture d'ecran
- Monde reel ? → Camera / Scanner
- Web ? → Telechargement
- Imagination ? → Generation IA
- Video ? → Extraction frames

2. Valider la qualite

- Verifiez resolution, format, et integrite
- Utilisez `PIL.Image.verify()` pour detecter les corruptions
- Controlez la taille des fichiers

3. Gerer les erreurs

- Try/except pour toutes les operations I/O
- Timeouts pour les telechargements
- Verification de `camera.isOpened()` avant lecture

4. Optimiser les performances

- Asynchrone pour telechargements multiples

- GPU pour generation IA
- Batch processing pour extraction video

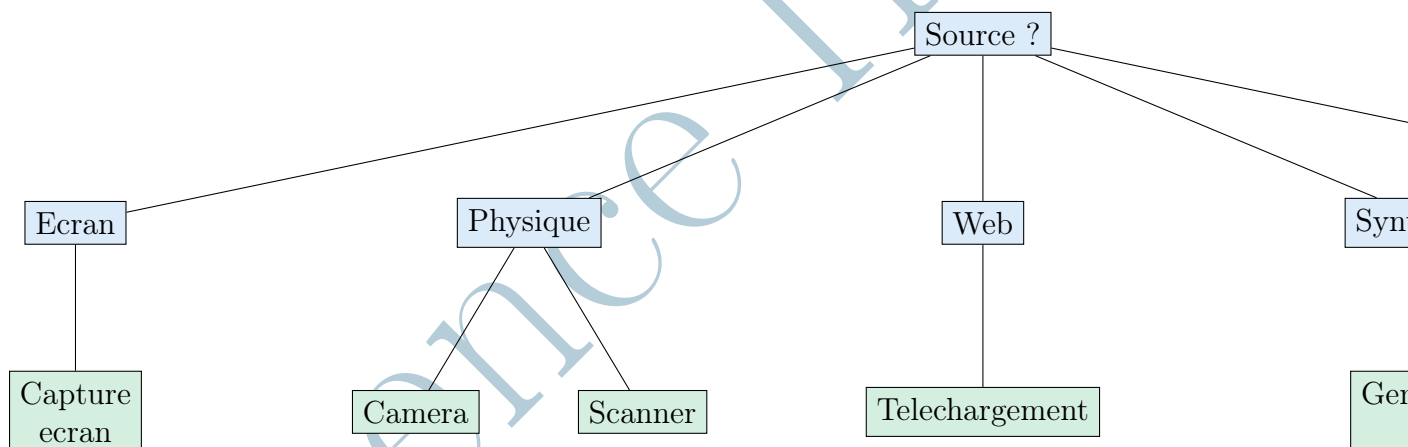
5. Respecter les standards

- PNG pour qualite sans perte
- JPEG pour compression avec perte acceptable
- WebP pour equilibre qualite/taille
- TIFF pour archivage professionnel

6. Documenter et versionner

- Timestamps dans les noms de fichiers
- Metadata (resolution, source, parametres)
- Logs d'acquisition

8.3 Arbre de Decision



Conseil

Pour un projet d'IA, combinez plusieurs mecanismes :

- Dataset initial : Telechargement d'images publiques
- Augmentation : Generation IA pour varier les donnees
- Donnees temps reel : Camera pour inference en production

Chapitre 9 Bibliographie et Ressources

9.1 Ouvrages de Reference

Pour approfondir vos connaissances, voici les ouvrages recommandes :

- Szeliski, R. (2022). *Computer Vision: Algorithms and Applications*, 2nd ed. Springer. — Reference complete en vision par ordinateur.
- Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press. — Fondations theoriques des modeles generatifs (GANs, VAEs).
- Rombach, R. et al. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models*. CVPR. — Article fondateur de Stable Diffusion.
- Ramesh, A. et al. (2022). *Hierarchical Text-Conditional Image Generation with CLIP Latents*. arXiv. — Article DALL-E 2.
- Saharia, M. et al. (2022). *Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding*. NeurIPS. — Article Imagen (Google).

9.2 Documentation Officielle

Outil	URL	Description
OpenCV	docs.opencv.org	Vision par ordinateur
Pillow	pillow.readthedocs.io	Manipulation d'images
Diffusers	huggingface.co/docs/diffusers	Stable Diffusion
OpenAI API	platform.openai.com/docs	DALL-E 3
MSS	python-mss.readthedocs.io	Capture ecran rapide
PyAutoGUI	pyautogui.readthedocs.io	Automatisation GUI
SANE	sane-project.org	Scanners Linux
TWAIN	twain.org	Scanners Windows/Mac

9.3 Tutoriels Recommandes

- **PyImageSearch** (pyimagesearch.com) — Tutoriels de qualite professionnelle :
 - OpenCV Basics and Advanced Techniques
 - Deep Learning for Computer Vision
 - Real-Time Object Detection
- **Hugging Face** (huggingface.co/docs) — Documentation complete sur Stable Diffusion et modeles generatifs.
- **OpenCV Tutorials** (docs.opencv.org/master/d9/df8/tutorial_root.html) — Guides officiels pour la capture video et le traitement d'images.

9.4 Jeux de Donnees pour Pratiquer

- **ImageNet** (image-net.org) — 14 millions d'images annotees, reference en classification.
- **COCO** (cocodataset.org) — Common Objects in Context, detection d'objets et segmentation.
- **Open Images** (storage.googleapis.com/openimages/web/index.html) — 9 millions d'images avec annotations riches.
- **LAION-5B** (laion.ai) — 5 milliards de paires image-texte, utilise pour entrainer Stable Diffusion.
- **Unsplash Dataset** (unsplash.com/data) — 2+ millions de photos haute qualite libres de droits.

9.5 Aspects Legaux et Ethiques

- **Droits d'auteur** — Verifiez toujours les licences avant d'utiliser des images telechargees. Creative Commons, domaine public, ou licences commerciales.
- **RGPD** — Attention aux donnees biometriques (visages) dans les captures video ou photos. Anonymisation requise dans certains cas.
- **Robots.txt** — Respectez les directives des sites lors du scraping d'images.
- **IA Generative** — Les images synthetiques peuvent poser des questions de propriete intellectuelle. Verifiez les CGU des modeles utilises.
- **Deepfakes** — L'utilisation malveillante de la generation d'images (usurpation d'identite) est illegale dans de nombreux pays.

Conseil

Pour un demarrage rapide :

- Capture ecran : `mss` (rapide et multi-plateformes)
- Camera : `OpenCV` (standard industriel)
- Telechargement : `requests` + `aiohttp` pour performance
- Generation IA : `Stable Diffusion` (gratuit, local) ou `DALL-E 3` (qualite maximale)